

HTML5 Arcade Architecture

First-Party Monitizer & AI Telemetry Prompt Sequence

IMPORTANT OPERATIONAL MANUAL: Upload this file directly to your AI system or copy segments sequentially. The landing hub is anchored to 'arcade.html'. Follow all 'STOP HERE' directives strictly to preserve architecture isolation before initiating actual game asset development.

System Deployment Map

This master engine provides automated layout population, standalone iframe security sandboxing, and data filtering layers without requiring hardcoded structural changes when expanding your catalog.

PHASE 1: DYNAMIC PORTAL LANDING HUB (ARCADE.HTML ARCHITECTURE)

You are an expert full-stack web developer. We are building the core infrastructure for an HTML5 Game Arcade Website. This project will scale to host hundreds of games on our own domain before we ever open it up to external site embeddings.

The system must be completely modular so that new games can be added to the portal by simply modifying a single `games.json` registry file, without touching the navigation bar, homepage layout, or routing code.

Please generate the code for the following three files to complete Phase 1:

- `games.json` (The Master Registry File):
 - Design a clean JSON array structure of game objects.
 - Each object must include: `id`, `title`, `category`, `description`, `thumbnail_url`, and `directory_path` (e.g., `"/games/rogue-delver/index.html"`).
- `arcade.html` & `style.css` (The Main Portal Landing Hub):
 - Note: The main landing/entry page for this arcade must be named `arcade.html`.
 - A premium, dark-themed responsive desktop and mobile gaming interface with a structural sidebar for categories and a main canvas grid.
 - The layout must feature a dedicated, isolated "Game View Container" holding a secure wrapper. Include a "Theater Mode / Fullscreen" toggle switch that controls container scaling.
- `portal.js` (The Client-Side Router & Dynamic Builder):
 - Write clean, vanilla JavaScript to fetch `games.json` immediately upon page initialization.
 - Automatically build the navigation categories and populate the homepage grid cards using the JSON data.
 - Implement Hash-Based Routing (e.g., `domain.com/arcade.html#/play/game-id`).

When a user selects a card, the application must intercept the URL hash change, hide the browse grid, display the Game View, and inject the clean path into the <iframe> src.

- Ensure the browser's native "Back" button cleanly updates the page state, unloads the iframe memory completely, and returns the user to the browse grid.

STOP HERE. Provide the complete code for Phase 1. Do not generate code for advertisements, telemetry pipelines, or game scripts yet. Ask me for "Phase 2" when you have verified this architecture is perfectly functional.</pre>

</div>

<div class="prompt-block">

<div class="prompt-header">PHASE 2: In-House Ad API Integration & Frame Bridging</div>

<pre>Excellent. Let's move to Phase 2: Internal Ad Network Wrapper & In-Game Event Hooks.

Because we are focusing strictly on maximizing ad revenue on our own site before opening up to external embeddings, we need to establish a highly reliable communication bridge between the game frames and our main parent site (`arcade.html`).

Please update `portal.js` and provide structural documentation executing the following:

1. Parent-Side PostMessage Orchestrator:

- Create an explicit `window.addEventListener("message")` listener on the parent site.

- The listener must capture event requests coming from the internal <iframe>, specifically looking for ad triggers (`ARCADE\_TRIGGER\_AD`).

2. Interstitial & Rewarded Video API Loops:

- When a game signals it needs an "interstitial" (level change) or a "rewarded ad" (unlocking a feature), it must halt the game iframe.

- Simulate the ad network's video sequence by deploying a modal overlay directly over the game frame with a 5-second structural countdown timer (this acts as a perfect drop-in placeholder for SDKs like Applixir or AdinPlay).

- Once the timer expires, the parent page must cleanly fire an asynchronous postMessage back to the target game frame: `iframe.contentWindow.postMessage({ type: "ARCADE\_AD\_COMPLETE" }, "\*")`.

3. Practical Implementation SDK Guide:

- Provide a clear Markdown guide detailing how to replace this simulation block with the real initialization scripts for Applixir or AdinPlay web-game SDKs inside our parent wrapper architecture.

STOP HERE. Provide the updated parent communication engine and the programmatic SDK guide. Do not write data telemetry layers or independent game code yet. Ask me for "Phase 3" when complete.</pre>

</div>

```
<div class="prompt-block">
  <div class="prompt-header">PHASE 3: Anonymized AI Telemetry Datasets &
Value-Exchange Guard</div>
  <pre>Perfectly handled. Let's build Phase 3: The Consenting AI Telemetry
Pipeline.
```

AI research labs are seeking data on human strategic decision-making, spatial reasoning, and cognitive paths. We will build a pipeline that lets consenting users share their anonymous gameplay telemetry (like step-by-step logic paths, choices, and delay timings). To maximize user opt-in, users who agree will have all third-party banner ads completely disabled for their session.

Please update our application architecture to implement the following:

1. The Explicit Consent Guard UI:

- Add a sleek, modern, unbundled privacy selector panel below the game frame container in `arcade.html`.
- Text: "Help train open-source AI models by sharing anonymous gameplay logic paths? In exchange, third-party tracking banner ads are completely disabled for this session."
- Save the player's configuration setting directly inside `window.localStorage` so it persists across page reloads.

2. Telemetry Ingestion Hub:

- In `portal.js`, build a message handler looking for `{ type: "ARCADE\_TELEMETRY\_STREAM" }`.
- If the user's local storage choice evaluates to TRUE, intercept the rich JSON data object (which will map frame states, coordinate selections, and calculation delays).
- Create a clean `streamDataToLogEngine(payload)` skeleton function utilizing a non-blocking `fetch(POST)` layout to demonstrate how these batches are streamed out to an API endpoint for database storage.
- If the user has opted out, the script must throw away/ignore all arriving payloads immediately at the browser level.

STOP HERE. Provide the complete code update for this data tracking implementation. Do not build individual game engines yet. Ask me for "Phase 4" when done.</pre></div>

```
<div class="prompt-block">
  <div class="prompt-header">PHASE 4: System Hardening, Liability Shields
& Data Filters</div>
  <pre>Excellent. Now let's implement Phase 4: Security Hardening, Liability
Protection, and Safety Infrastructure. This is vital to protect us legally regarding
the data pipeline and secure our server assets before writing actual games.
```

Please configure the framework with these explicit defense profiles:

1. Strict Iframe Security Sandboxing:

- Update our `arcade.html` layout so the game iframe container utilizes a highly restrictive, secure sandbox configuration: `sandbox="allow-scripts allow-same-origin"`.

- Ensure the iframe tag includes strict hardware permissions: `allow="autoplay; fullscreen"`. This ensures games can safely request fullscreen states and play ad audio natively without ever accessing parent DOM controls or cookies.

2. Client-Side Telemetry Guard Filters:

- Build a structural sanitize/validation filter directly inside the `portal.js` telemetry receiver.

- Before forwarding any JSON data packet to our mock server streaming pipeline, verify the payload structure. Strip out or throw an error if the payload contains any unauthorized keys or potential text input values (ensuring no PII, emails, names, or malicious script injections can pass through our data pipeline to the database).

3. Front-Facing Transparency Documentation (Terms & Privacy Templates):

- Provide clean, plain-text template blocks to display on our site's legal pages:

A. An "AI Telemetry and Data Use Notice" detailing that data collection is 100% anonymized behavioral coordinate mapping with zero personal identifier hooks, fulfilling strict compliance guidelines (like the EU AI Act data collection standards).

B. A "Limitation of Liability & User-Generated Content Disclaimer" protecting the site operator from any unexpected browser crashes, server bandwidth latency, or local data clearing.

STOP HERE. Deliver the fully hardened application files and the legal framework documentation templates. Do not proceed to game development yet. Ask me for the final step when you are done.</pre>

</div>

<div class="prompt-block">

<div class="prompt-header">PHASE 5: Sandbox Engine Client Deployment Template</div>

<pre>The entire foundational arcade platform architecture is rock-solid and secure. We are ready for Phase 5: Building our first fully framework-compliant game template.

Please construct the game "Grid Delver (The 1-Minute Micro-Rogue)" as a self-contained web asset directory (`index.html`, `style.css`, and `game.js`). It must run perfectly inside our sandbox environment.

Incorporate these mandatory functional features:

1. Turn-Based Game Engine Loop:

- Build a minimalist, crisp 5x5 grid puzzle using Canvas or modular DOM nodes. The player moves card-by-card from a starting node to discover traps, hidden monsters, gold, and stairs.

2. Asynchronous Ad Hook Trigger:

```

- Upon player death or when moving to a new level, the game engine must call a
local pause state (suspending game tickers and Web Audio contexts) and fire:
`window.parent.postMessage({ type: "ARCADE_TRIGGER_AD", adType: "interstitial" },
"*")`.

- Implement an event listener waiting for the parent's confirmation payload:
`type: "ARCADE_AD_COMPLETE"`. Only upon receiving this back can the game loop resume
seamlessly.

3. Granular High-Value AI Telemetry Engine:

- The game loop must track the timeline of every single turn. For each move made
by the player, record an immutable object containing: player status, visible grid
layout indices, destination coordinates selected, and the exact thinking time
elapsed in milliseconds ( $T_{\text{delta}} = T_{\text{click}} - T_{\text{state\_ready}}$ ).

- Every 5 moves, compile these objects into a single batch and securely stream it
upwards via: `window.parent.postMessage({ type: "ARCADE_TELEMETRY_STREAM", payload:
telemetryBatch }, "*")`.

Deliver the complete, independent game folder code files, ensuring they are cleanly
written, production-ready, and detached from our main site core logic.</pre>
</div>

</body>
</html>

```